

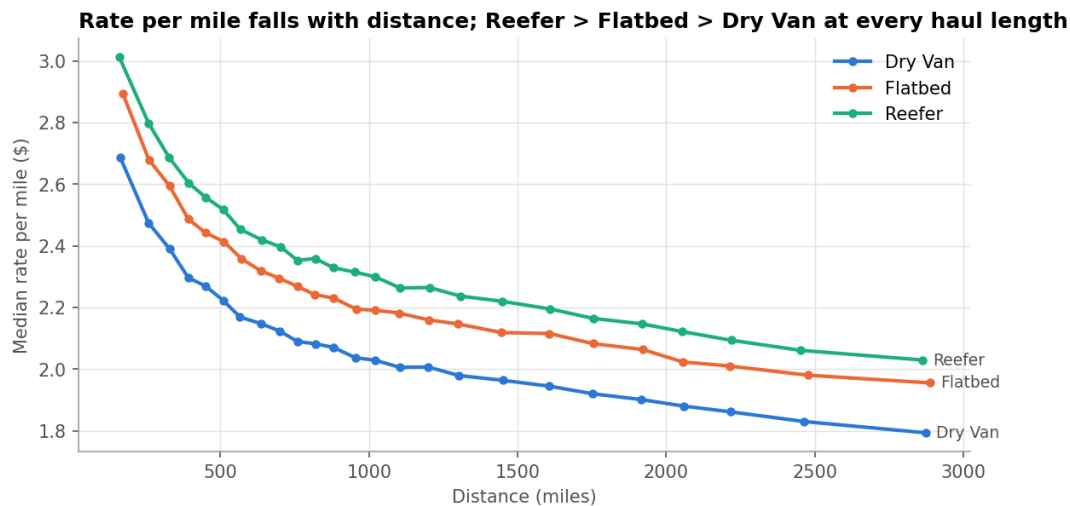
Freight Rate Prediction

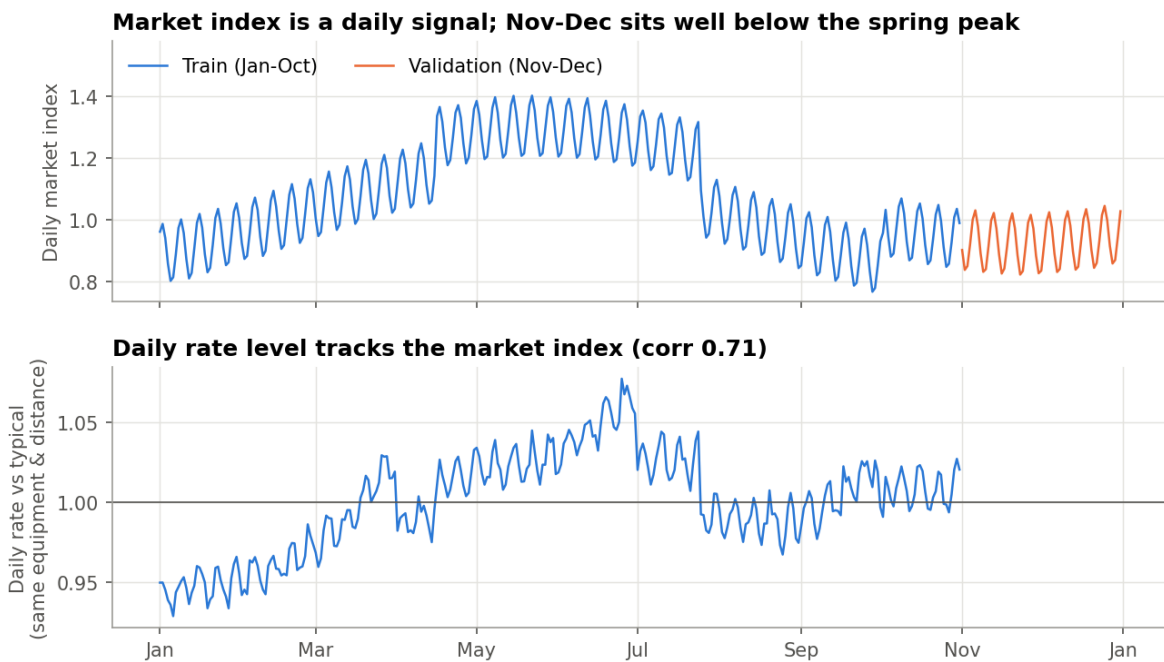
Machine Learning Engineer assessment - approach, validation and results

Question	Answer
Task	Predict posted_rate for 12,000 loads in Nov-Dec 2025 from 48,000 labelled loads (Jan-Oct 2025)
Split	Time-based, expanding window; each fold predicts the next two months (no random split)
Model	$\log(\text{rate}) = \text{linear market/calendar part} + \text{LightGBM (linear trees) on load features} + \text{shrunk lane premium}$
Accuracy on unseen months	1.50% MAPE, \$35.6 MAE (rate-card baseline: 4.15%, \$96.0); 89% of loads within 3%
Key insight	Rates rise 2-8% over the last month of every quarter (strongest for Flatbed); December is a quarter end

1. Data and key findings

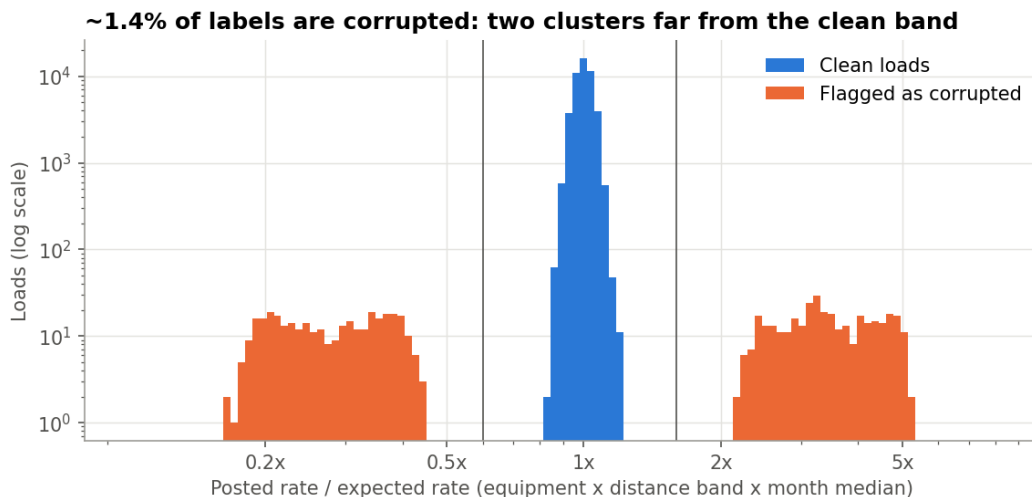
- **Scoring data is the future.** train_test.csv covers 2025-01-01 to 2025-10-31; validation.csv covers 2025-11-01 to 2025-12-31. This is a forecasting problem, which drives the whole validation design.
- **Pricing structure.** Rate per mile falls with distance ($\text{rate} \sim \text{distance}^{0.87}$). Reefer is ~13% and Flatbed ~8% above Dry Van at every distance; ~+3% per 10,000 lb.
- **Market index is a daily signal** (within-day std 0.025) with a strong weekly cycle (Thursday peak, Sunday trough). Its daily mean correlates 0.71 with the daily rate level; a 10% higher index adds ~1.5%.
- **Quarter-end ramp.** After removing distance, equipment, weight and market effects, rates are flat for two months of each quarter, then climb ~5% over the last month and reset when the next quarter opens. Seen in Q1, Q2 and Q3 alike.
- **quote_signal is a trap.** Against a log-linear model it shows a +/-3% U-shape; against a flexible model the effect is under 0.3%. The shape came from short hauls, where quote_signal is much more spread out.





2. Data-quality issues

Issue	Size	How it was handled
Corrupted posted_rate labels	677 (1.41%)	Ratio to a robust expected rate (median \$/mi per equipment x distance band x month). Clean loads sit in 0.8-1.2x; corrupted ones in 0.16-0.45x and 2.1-5.2x with nothing in between. Cut at 0.6x / 1.6x, removed from training.
Negative weights	292 train, 145 val	Sign-flip errors (they price like the absolute value): abs() + flag
Weight capped at 47,500 lb	1,191	Kept, flagged
Missing weight	300 / 165	Training median + flag
Missing market_index	374 / 249	Mean of all loads on that date (daily signal)
Synthetic coordinates	all 72 cities	Shifted but consistent per city; used as relative location only
Cities only in validation	8 cities, 12% of val loads	No city or lane IDs in the trees; tested by hiding 8 cities: 1.9% vs 1.7% error
December inputs lack market_index and coordinates	31 rows	Coordinates from the city table; market index = that day's mean in validation.csv (features only)



3. Validation and train/test split

Why not a random split. The loads to predict come from the two months after the labelled data. A random split would put November-like days (and their market states) on both sides and reward models that memorise days, overstating accuracy. Every fold therefore trains on an expanding window and tests on the next two months only.

Fold	Train	Test	Purpose
Q2	Jan - Apr	May - Jun	Same shape as the real task: train through month 1 of a quarter, predict months 2-3 (Oct -> Nov-Dec)
Q3	Jan - Jul	Aug - Sep	Same shape as the real task
X1	Jan - Jun	Jul - Aug	Stress test: predict into a quarter never seen
X2	Jan - Aug	Sep - Oct	Most recent two months

- **Label cleaning inside the split.** Corrupted labels are removed from training rows only. Metrics are reported on clean test labels (true model accuracy) and on all test labels (what a noisy scoring set shows).
- **Number of trees.** Early stopping on the latest 20% of each training window (time-ordered, never the test months), then refit on the full window with 15% more rounds.
- **Model selection** used the mean over all four folds and required gains to hold on the two production-like folds; changes that only moved error between folds were rejected (e.g. splitting the market index into smooth and weekly parts).
- **Unseen cities.** 8 random cities hidden from training (3 draws): error on loads touching them 1.85-2.05% vs 1.68% for known cities.
- **Uncertainty.** 90% range of -2.7% / +3.0% around the prediction, calibrated on Q2+Q3 and checked on the other folds (89.8% coverage on Sep-Oct; 77% when predicting into an unseen quarter).
- **Leakage checks.** No feature uses the target; lane premiums use out-of-fold residuals inside the training window; validation.csv contributes only features (cities, daily market index), never labels. Unit tests assert that no fold trains on dates after its test window.

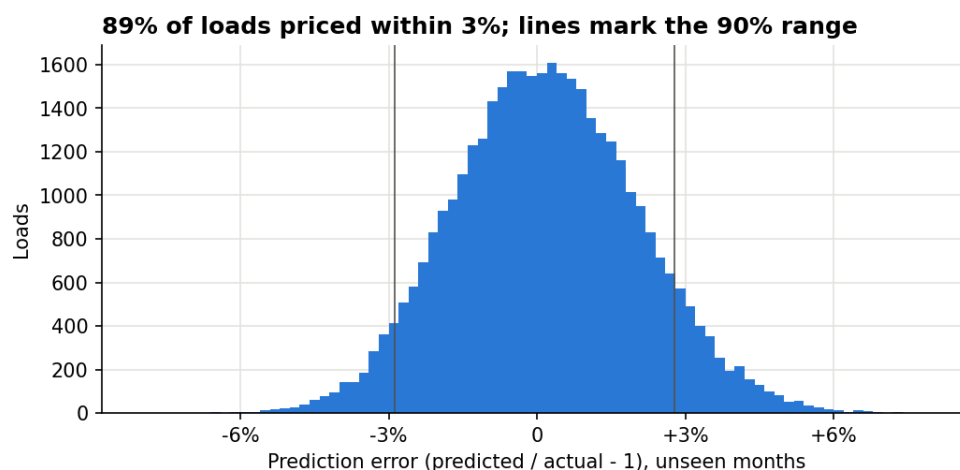
4. Model

$\log(\text{rate}) = \text{market part}$ (linear: market index elasticity, quarter-end ramp by equipment, monthly drift) + **load part** (LightGBM with linear trees on distance, equipment, weight, coordinates, circuitry and weight flags) + **lane premium** (out-of-fold residual per lane, shrunk by $n/(n+10)$). Five seeds are averaged.

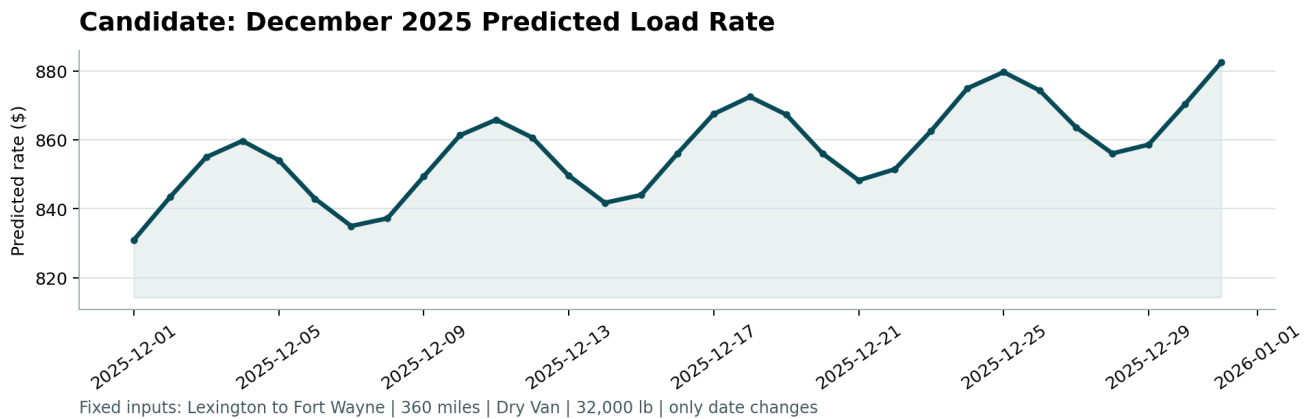
- Plain LightGBM was unstable across folds: the daily market value is unique per day, so the trees used it as a date ID and memorised day-level prices; with a time feature they held the last level flat. Moving all time-varying effects into a small linear part lets them extrapolate, and the trees only learn the stable 'what does this load cost' structure.
- Linear trees fit the smooth distance/weight curves better than flat leaves (1.62% -> 1.53%).
- Remaining error: a second model trained to predict the final model's errors from all load features explains only ~3% of them, so what is left is close to the noise in negotiated spot prices.

Model	Q2	Q3	X1	X2	Mean MAPE	Mean MAE
Rate-card baseline (median \$/mi by equipment x distance)	6.04%	3.47%	3.62%	3.48%	4.15%	\$96.00
Log-linear regression + trend	1.92%	1.92%	2.27%	1.98%	2.02%	\$46.45
LightGBM, no time feature	1.84%	3.01%	3.03%	2.95%	2.71%	\$61.16
LightGBM, time as a feature	2.24%	2.31%	2.40%	1.93%	2.22%	\$50.62
LightGBM, detrended, without label cleaning	2.82%	3.14%	3.39%	2.20%	2.89%	\$65.38
LightGBM, detrended	1.54%	2.81%	2.98%	1.55%	2.22%	\$52.10
Hybrid: linear market part + standard trees	1.51%	1.67%	1.96%	1.49%	1.66%	\$38.12
Hybrid with linear trees	1.42%	1.39%	1.87%	1.45%	1.53%	\$36.32
FINAL: hybrid, linear trees, 5 seeds, lane premium	1.39%	1.35%	1.84%	1.42%	1.50%	\$35.64

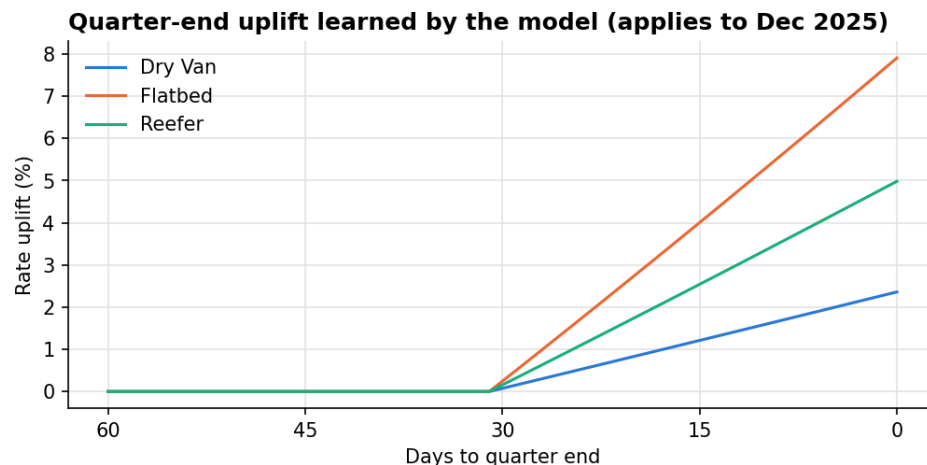
MAPE on clean test labels per fold. Including the 1.4% corrupted test labels, MAPE is ~3.9% and RMSE ~\$620 for every reasonable model: those rates are 2-5x off and cannot be predicted.



5. Fixed December prediction chart (score.py)



- Inputs are fixed (Lexington -> Fort Wayne, 360 mi, Dry Van, 32,000 lb); only the date changes. The file has no market index or coordinates, so the FeatureBuilder fills coordinates from the city table and the market index from that day's mean across validation.csv loads.
- **Weekly zig-zag:** the market index's weekly cycle (Thursday peaks).
- **Upward drift (\$831 -> \$882, +6%):** the quarter-end / year-end ramp plus the underlying monthly drift.
- **Sanity check:** real Dry Van loads on this lane in training ranged \$758-\$934 (e.g. \$864 on 30 Oct at market index 1.01); the December predictions are consistent with that history.



6. Business use and limitations

- **Quarter-end pricing:** on the last day of a quarter the model adds ~\$164 to a median Flatbed load, ~\$110 to Reefer and ~\$46 to Dry Van.
- **Quote ranges:** the 90% band (-2.7% / +3.0%) gives a floor and ceiling for negotiation.
- **Data guardrail:** the ratio check that found the corrupted labels can flag suspicious rates at entry.
- **Limitations:** no holiday effect can be learned from Jan-Oct, so Christmas week is priced at its market level (no holiday effect was visible around Memorial Day, July 4 or Labor Day either); the year-end ramp is assumed to behave like Q1-Q3, and the last 1-3 days of past quarters were under-predicted by ~0.5-1%; in production the daily market index would come from the market feed rather than from the scoring file.

Code, run instructions and a step-by-step analysis log: see the repository README and docs/ANALYSIS_LOG.md. The whole submission is reproduced with `python run_all.py`.